

전총명

Backend Developer · Cloud · AI Integration

010-8362-0913 zsc0248@naver.com github.com/Kade-Jeon

ABOUT ME

운영 환경의 문제를 구조 개선으로 해결합니다

저는 레거시 구조를 이해하고, 운영 환경의 문제를 구조 개선으로 해결하는 Java 백엔드 개발자입니다. 운영 중인 서비스에 서는 작은 버그 하나도 사용자 경험과 운영 비용에 직결된다고 생각하기 때문에, 단순 수정에 그치지 않고 원인을 구조와 흐름에서 찾으려 노력해 왔습니다.

크리니티에서 공공·클라우드 메일 서비스를 개발·운영하며 고객사 기능 개발, 운영 이슈 대응, 모니터링 체계 구축, AI 기능 통합 PoC 설계 등을 경험했습니다. 이 과정에서 **AI 호출 구조 개선**, **통합 모니터링 구축**, **SMS/MMS 구조 재설계**를 통해 서비스를 더 유지보수하기 쉽고 확장 가능한 방향으로 개선하는 데 집중했습니다.

저의 강점은 **새로운 기술을 무조건 도입하는 것이 아니라**, 현재 서비스와 조직의 운영 방식에 맞는 현실적인 선택을 하고 이를 **운영 가능한 구조로 구현**하는 데 있습니다.

WORK EXPERIENCE

경력 사항

크리니티

2024.05 — 2026.01 · 약 1년 9개월

Backend Developer · Cloud

담당 업무

- 공공·클라우드 메일 서비스의 기능 유지보수 및 운영 안정화
- 고객사 맞춤 기능 개발 및 JavaScript · JSP 기반 서비스 구현
- Spring AI 기반 AI 기능 통합 PoC 설계 · 개발
- 검색 기능 개선 및 Elasticsearch 기반 성능 최적화 참여
- ELK · Fleet · APM 기반 통합 모니터링 환경 구축 및 운영

주요 프로젝트 / 성과

01 Spring AI 기반 공공 메일 서비스 AI 기능 PoC

상황

레거시 메일 서비스에는 이미 AI 호출 기능이 있었는데, RestTemplate으로 LLM API를 직접 호출하는 방식이었습니다. 시스템 프롬프트·사용자 메시지·컨텍스트가 구분 없이 하나의 문자열로 합쳐져 전송되다 보니 AI 응답 품질이 낮았고, RAG·에이전트 구조로 확장하는 것도 구조적으로 불가능한 상태였습니다.

행동

회사 내 Java/Spring 기술 스택에 대한 선호가 강한 점을 감안해, Python 계열 프레임워크(LangChain 등) 대신 Spring AI 도입을 직접 제안했습니다. ChatClient 구조로 시스템 프롬프트·메시지 역할을 명확히 분리하고, 1차로 메일 본문 번역·외국어 메일 작성 기능을 구현했습니다. 이후 RAG 파이프라인, 메일·주소록 도메인 단위 에이전트 구조, Langfuse 기반 Observability로 확장할 수 있도록 아키텍처를 설계했습니다.

- 결과** 한국지능정보사회진흥원(NIA) 대상 번역·메일 작성 AI 기능 베타 운영 도입 (2026.01~). ChatClient 역할 분리 구조 전환 후 응답 일관성이 눈에 띄게 향상되었습니다. G-cloud 공공 클라우드 메일의 최초 AI 기능으로 채택되었고, 이후 RAG 기능 추가 확장 요청이 들어왔지만 사전 설계 덕분에 재설계 없이 대응 가능한 상태입니다.
- 배운 점** 아무리 프롬프트 엔지니어링을 반복해도 응답 품질이 오르지 않는 상황에서, 문제의 원인이 프롬프트 내용이 아니라 구조 자체에 있었음을 파악했습니다. 기능을 동작하게 만드는 것과 확장 가능하게 설계하는 것은 다른 문제임을 실감했고, 사전에 확장을 고려한 설계 덕분에 이후 RAG 기능 추가 요청에 재설계 없이 대응할 수 있었습니다.

02 KISTI 프라이빗 클라우드 ELK 모니터링 시스템 구축

- 상황** 기존에는 Scouter로 일부 메트릭만 확인 가능했고, 로그를 보려면 관련 인원이 모든 VM에 root로 접속해 분산된 로그를 직접 확인해야 해서 장애 원인 파악이 느리고 보안·운영 리스크가 있는 상태였습니다.
- 행동** Docker 기반 ELK Stack(Elasticsearch·Logstash·Kibana)을 구성하고, Fleet + Elastic Agent로 중앙 수집, Admin/Readonly 권한 분리, Elastic APM으로 Java 메일 서비스의 성능·트랜잭션 모니터링을 구성했습니다.
- 결과** 한국과학기술정보연구원(KISTI) 프라이빗 클라우드에 도입 (2025.02~). VM root 접속을 필수 인원으로 제한하고 그 외 인원은 Kibana를 통해 로그를 조회하도록 전환해 보안을 강화했고, 중앙 집중 조회로 장애·CS 대응 시간을 기존 대비 약 30% 수준으로 단축했습니다. Elastic APM 슬로우쿼리 로그에서 수신했던 기능의 메일 조회 SQL이 반복 감지되는 것을 발견해 is_spam/is_deleted 의미 없는 필터 조건을 제거, 쿼리 성능을 개선했습니다.
- 배운 점** 모니터링 가시성이 생기자 기존에는 누구도 인지하지 못했던 슬로우쿼리가 보이기 시작했고, 이를 발견해 직접 개선할 수 있었습니다. 문제는 보여야 해결할 수 있다는 것, 그리고 불편함에서 끝내지 않고 해결 방향을 제시하는 것이 개발자의 역할임을 확인했습니다.

03 레거시 검색 구조 기반 결과 내 재검색 기능 구현

- 상황** 기존 검색 시스템은 일반 검색과 상세 검색 모두 1회성 조회만 가능해, 사용자가 검색 결과를 단계적으로 좁혀가는 흐름을 지원하지 못했습니다. 사내 검색 엔진은 과거 Lucene 기반 코드를 포크해 온 구조를 그대로 사용하고 있어 해당 기능을 직접 지원하지 않았고, 검색 영역 자체도 쉽게 수정하기 어려운 상태였습니다.
- 행동** 검색 엔진 자체를 수정하는 대신, 초기 검색 결과 집합과 재검색 결과 집합의 교집합만 최종 조회 대상으로 반영하는 방식으로 로직을 설계했습니다. 1차 검색 결과의 mailUid 목록을 기준으로 재검색 요청 시 두 결과 집합의 교집합만 남긴 뒤 해당 결과만 DB 조회해 반환하도록 처리했습니다.
- 결과** 결과 내 재검색 기능을 도입하면서 현재 검색 결과를 기준으로 최대 5회까지 단계적으로 범위를 정제할 수 있게 되었고, 메일이 많이 누적되는 업무용 환경에서도 원하는 메일에 더 빠르게 도달할 수 있는 검색 흐름을 제공했습니다.
- 배운 점** 레거시 환경에서는 완전히 새로운 구조를 도입하는 것보다, 기존 동작을 정확히 이해하고 손댈 수 있는 범위 안에서 기능을 확장하는 판단도 중요하다는 점을 배웠습니다.

04 레거시 공통 히스토리 처리 정상화 및 네비게이션 상태 개선

- 상황** 기존 메일 서비스는 사내 공통 JavaScript 패키지에서 별도로 히스토리 이벤트를 처리하는 구조였습니다. 이 때문에 메일 상세 화면에서 목록으로 돌아온 뒤 다시 뒤로가기를 하면 직전 메일 화면으로 복귀하지 못하고, 좌측 메일함 사이드바의 하이라이트 상태가 실제 화면과 어긋나는 문제도 발생했습니다.
- 행동** 사내 패키지의 히스토리 처리 로직부터 다시 분석한 뒤, 메일 상세/메일함 목록/다른 메일함 이동 과정에서 뒤로가기 시점마다 어떤 화면 상태와 하이라이트 상태가 유지되어야 하는지를 기준으로 흐름을 재정의하고, 히스토리 처리와 UI 상태 동기화 로직을 함께 수정했습니다.
- 결과** 메일 상세/목록/메일함 이동 흐름이 사용자가 기대하는 방식으로 일관되게 동작하도록 개선했습니다. 개선 이후 뒤로가기 관련 문의도 0건으로 줄어, 반복적으로 발생하던 사용자 혼란을 해소할 수 있었습니다.
- 배운 점** 레거시 환경에서의 작은 사용성 문제도 실제로는 공통 패키지와 전역 상태 처리 방식에 깊게 연결되어 있을 수 있다는 점을 배웠습니다.

05 고객사 SMS/MMS 외부 API 연동 개발

- 상황** 클라우드 메일 서비스 고객사에서 기존 SMS 기능 외에 MMS(이미지 첨부 발송) 기능 추가를 요청했습니다. 기존 SMS 로직이 확장을 고려하지 않은 구조여서, 그대로 MMS를 얹으면 기존 기능 안정성에 영향을 줄 우려가 있었습니다.

행동

기존 SMS가 shell curl 기반으로 통신사 API를 호출하고 있어, 통신사 최신 API 문서를 기반으로 RestTemplate 방식으로 재구현했습니다. 공통 발송 로직과 타입별 처리 로직을 분리해 재설계하고, 엑셀 대량 발송 기능을 추가하며 CompletableFuture로 병렬 발송 후 전체 결과를 집계해 반환하도록 구현했습니다.

결과

한국농수산물유통공사 운영에 적용되었습니다 (2025.06~). 엑셀 기반 대량 발송 도입으로 기존 건별 수동 등록 방식 대비 발송 준비 시간이 크게 단축되었고, SMS 단독 운영에서 MMS까지 발송 가능한 콘텐츠 유형이 확대되었습니다.

배운 점

shell curl로 작성된 레거시 SMS 코드를 RestTemplate으로 재구현하면서, 기존 코드가 왜 그렇게 작성됐는지 파악하는 것부터 시작해야 했습니다. 통신사 API의 수신자 제한이라는 외부 제약을 설계에 반영해 체크 분할·병렬 처리 구조로 대응했습니다.

사용 기술

Spring Framework

Spring Boot 3.x

Java

MySQL

Kafka

Redis

ElasticSearch

ELK Stack

Spring AI

Langfuse

KT Cloud

NCP

Docker

Jenkins

JSP

JavaScript

PROJECTS

프로젝트

포트폴리오

AI Assistant — Backend

Spring Boot 3.5 · Java 21 · Spring AI 1.1.2

Java/Spring 기반으로 운영 가능한 AI 어시스턴트 백엔드 구조를 검증한 프로젝트. RAG · SSE 스트리밍 · 대화 메모리 · 관측성 · 프롬프트 관리까지 포함해 실제로 운영 가능한 AI 백엔드 구조를 목표로 설계했습니다.

프로젝트 목적

단순 챗봇 데모가 아니라, **AI 기능을 기존 Java/Spring 백엔드 서비스 안에 넣었을 때 필요한 구조를 검증**하는 것이 목적입니다. RAG, SSE 스트리밍, 대화 메모리, 관측성, 프롬프트 관리까지 포함해 실제로 운영 가능한 AI 백엔드 구조를 목표로 설계했습니다.

왜 Spring AI인가 (Python이 아닌 이유)

Python 생태계(LangChain 등)가 더 성숙하다는 점은 알고 있었습니다. 하지만 이 프로젝트의 목적은 **모델 실험이 아니라 서비스 통합**이었습니다. 메모리 관리, 스트리밍, 관측성, 장애 대응 같은 문제를 **Java/Spring 백엔드 개발자 관점**에서, 기존 서비스 구조 안에서 풀어보고 싶었습니다.

검증 결과

로컬 Ollama와 연동해 **RAG · SSE 스트리밍 · 대화 메모리 · Langfuse 추적**이 **end-to-end로 동작**함을 확인했고, Java 백엔드 안에서 운영 가능한 AI 구조를 설계할 수 있다는 점을 검증했습니다.

핵심 설계 포인트

SSE 실시간 스트리밍

토큰 단위 스트리밍으로 응답 대기 체감을 줄이고, 재시도 구조를 적용해 실제 서비스에서의 응답 안정성까지 고려했습니다.

Tool-based RAG

프로젝트형 채팅방에서만 벡터 검색 툴이 동작하도록 분리해, 불필요한 검색 비용을 줄이고 필요한 경우에만 근거 기반 답변을 제공하도록 했습니다.

Idempotency 보장

중복 요청과 재시도 상황에서도 동일한 응답이 여러 번 저장되지 않도록 상태 기반 처리 구조를 설계했습니다.

LLM 완전 추적

이중 캐시 메모리

동적 프롬프트 관리

프롬프트·응답·지연시간을 추적 가능하게 구성해 운영 중 품질 문제와 지연 원인을 분석할 수 있도록 했습니다.

Redis와 PostgreSQL에 대화 이력을 이중화해 응답 속도와 데이터 내구성을 함께 확보했습니다.

프롬프트와 모델 파라미터를 외부에서 관리할 수 있도록 분리해, 배포 없이도 품질 개선과 실험을 반복할 수 있도록 했습니다.

기술 스택

Spring Boot 3.5

Java 21

Spring AI 1.1.2

Ollama

pgvector

Redis

PostgreSQL

OpenTelemetry

Langfuse

Docker

핵심 아키텍처 결정 (ADR)

ADR-001 Tool-based RAG vs Naive RAG

매 요청마다 벡터 검색을 수행하면 응답 지연·비용 증가. LLM이 컨텍스트 필요 여부를 판단해 Tool을 선택 실행하도록 설계 → 평균 응답 지연 최소화 + OCP 준수

ADR-002 Idempotency — Strategy 패턴

SSE 환경에서 Key 상태(없음/진행중/완료/실패)에 따라 처리 로직이 완전히 달라짐. 4가지 Handler 구현체로 상태별 캡슐화 → SRP + OCP

ADR-003 Redis L1 + PostgreSQL L2 이중 메모리

DB 단독 → 응답 지연, Redis 단독 → 재시작 시 맥락 소실. 이중 레이어로 속도와 내구성 동시 확보. Spring AI 인터페이스 직접 구현으로 중복 메시지 방지

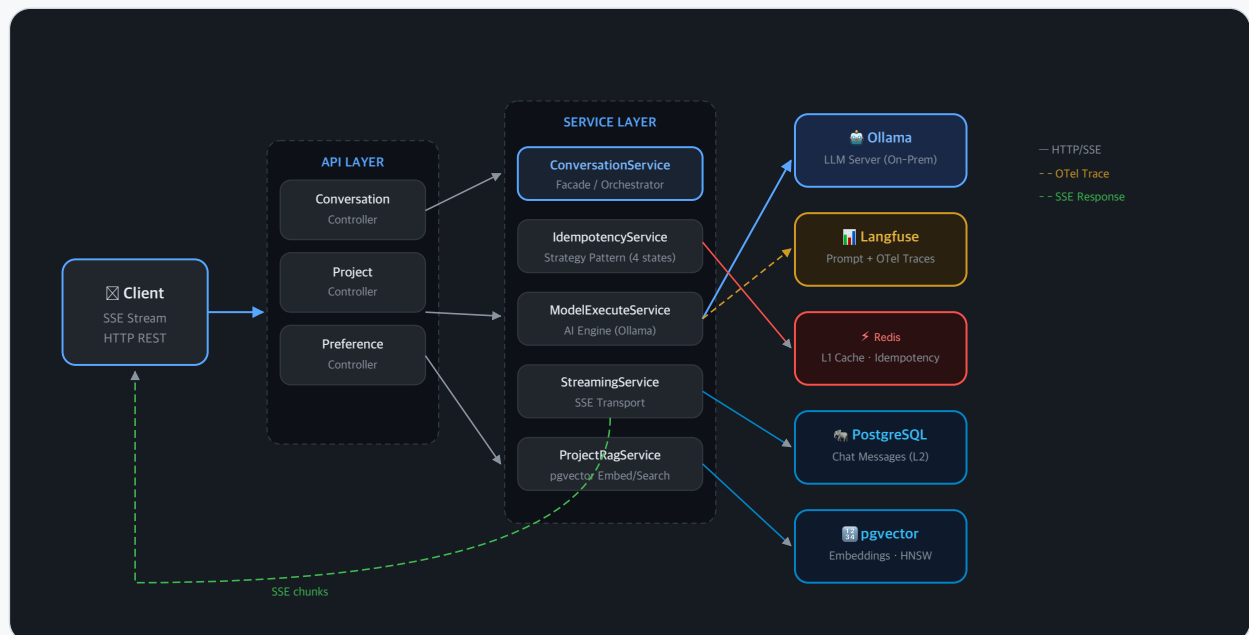
ADR-004 Java 21 Virtual Thread 전면 활성화

SSE 다중 동시 연결 + Ollama I/O 블로킹.
spring.threads.virtual.enabled=true 설정만으로 OS 스레드 수 상관없이 높은 동시 처리 달성

ADR-005 Langfuse 동적 프롬프트 관리

프롬프트를 코드에 하드코딩하면 모델 교체·튜닝 시 재배포가 필요. Langfuse API에서 런타임 조회 + Redis 1시간 TTL 캐시 + OpenTelemetry OTLP로 요청 단위 추적까지 통합

시스템 아키텍처



트러블슈팅

ChatMemoryRepository 커스텀 — 안정적 ID로 첨부파일 FK 연관관계 구현

- 문제** Spring AI 기본 JdbcChatMemoryRepository는 메시지 저장 시 대화 전체를 **삭제 후 재삽입**하여 CHAT_MESSAGE ID가 매 요청마다 재생성. CHAT_ATTACHMENT와 FK 연관관계를 맺는 것이 불가능했습니다.
- 해결** ChatMemoryRepository 인터페이스를 직접 구현하여 기본 자동 구성 비활성화 및 교체. saveAll()을 **delete+insert → append-only** 방식으로 재정의. type + content 중복 검사 후 신규 메시지만 INSERT하고, 엔티티 생성 시 UUID.randomUUID()로 ID를 1회만 할당하여 불변성 보장.
- 결과** CHAT_MESSAGE ID 안정화로 FK · JPA @OneToMany 매핑 정상 동작. OCP 원칙에 따라 기존 코드 수정 없이 구현체 교체만으로 해결.

X-Idempotency-Key 기반 중복 요청 방지 — 네트워크 재시도 시 데이터 일관성 확보

- 문제** SSE 스트리밍 환경에서 클라이언트 재시도 시, 동일한 사용자 메시지가 DB에 중복 저장되고 AI 모델 호출이 중복 발생하여 자원이 낭비되는 문제.
- 해결** X-Idempotency-Key를 도입하고 Redis를 활용해 요청 상태(IN_PROGRESS / COMPLETED / FAILED)를 관리. 재시도 시 기존 conversationId와 userMessageId를 재사용하여 DB 저장 로직을 스킵하도록 설계.
- 결과** 중복 메시지 발생 0% 달성 및 불필요한 AI 추론 비용 절감. 네트워크 장애 상황에서도 데이터 일관성 100% 보장.

AI Assistant Web — Frontend

Vue 3.5 · TypeScript 5.9 · Vite 7 · Tailwind CSS 4

백엔드 AI Assistant API와 연동하는 Vue 3 프론트엔드를 단독으로 구현했습니다. POST 기반 SSE 스트리밍 수신을 위해 EventSource 대신 Fetch API + ReadableStream을 직접 사용했고, Pinia 없이 Composables 패턴으로 상태를 관리했습니다.

핵심 설계 포인트

SSE 스트리밍 수신

POST 기반 SSE 구현에 EventSource 대신 Fetch API + ReadableStream을 직접 사용해 서버 토큰 스트림을 실시간으로 반영했습니다.

Composables 상태 관리

전역 스토어 없이 채팅·인증·대시보드처럼 관심사별로 Composable을 분리해 상태를 관리했습니다.

AI 사용 대시보드

Langfuse 통계 API를 연동해 사용량, 비용, 모델별 점유율을 시각화했습니다.

Vue Router 없는 SPA

currentView ref 기반으로 화면 전환을 직접 설계해 불필요한 번들 크기와 구조 복잡도를 줄였습니다.

마크다운 렌더링

AI 응답을 markdown-it으로 스트리밍 중에도 실시간 렌더링되도록 구현해 코드블록, 표, 목록을 지원합니다.

프로젝트 단위 채팅

일반 대화와 문서 기반 질의를 분리하기 위해 프로젝트 단위 채팅 구조를 도입해 독립된 컨텍스트를 유지합니다.

기술적 의사결정 (ADR)

FE-001 axios 없이 Fetch API 직접 사용

EventSource는 GET 요청만 지원 → POST 기반 SSE에 사용 불가. axios는 브라우저에서XHR 기반으로 동작해 response.body.getReader()를 지원하지 않음. fetch + ReadableStream이 POST 기반 SSE의 브라우저 표준 구현 방식.

FE-002 Pinia 없이 Composables 패턴으로 상태 관리

useChatState 호출부가 ChatPage.vue 단 하나라 전역 공유가 불필요. 각 페이지가 독립적으로 동작하며 라우트 간 공유 상태가 없음. useChatState 내부 onMounted 패턴은 Pinia 스토어와 근본적으로 맞지 않아 Composable 패턴이 더 적합.

기술 스택

Vue 3.5 TypeScript 5.9 Vite 7.2 Tailwind CSS 4 Shadcn-vue Fetch API SSE (ReadableStream) markdown-it

TECH STACK

기술 스택

Backend

Java 21 Spring Boot 3.x
Spring Framework Spring AI
Spring Data JPA

Database · Messaging

MySQL PostgreSQL Redis
Kafka pgvector

AI / LLM

Ollama RAG (Tool-based)
Spring AI ChatClient Embedding

Observability · 모니터링

ELK Stack Elasticsearch
Elastic APM OpenTelemetry
Langfuse

Cloud · Infra

KT Cloud NCP Docker
Jenkins Git

Frontend

Vue 3 TypeScript
Tailwind CSS 4 Vite JSP
JavaScript

CONTACT

함께 일해요

새로운 기회와 도전을 환영합니다. 언제든지 연락 주세요.

Email

zsc0248@naver.com

GitHub

github.com/Kade-Jeon

Wanted

wanted.co.kr/profile